

E-Closet Search V6 — Intent-Gated Cascade Architecture

LLM gold-standard attributes, structured-first retrieval, scroll-feed assembly

RecSys Team

2026-06-04

Executive Summary

E-Closet search is moving from a single fused-vector hybrid ranker to an **intent-gated cascade**. A lightweight query-understanding step routes each query; structured attributes extracted by an LLM (color, category) become hard pre-filters over trusted fields; dense embeddings rank the survivors and power the conceptual tail; and the result sections are concatenated into one scrollable feed rather than blended into a single fused score.

This design is driven by three findings:

1. **Sellers cannot be trusted to label attributes.** Listings over-tag color, occasion, and style for exposure. A per-item vision-LLM pass produces a reliable, normalized attribute set — the gold standard the search layer should rank on. A 500-item proof of concept confirmed this is accurate and cheap (\$0.08 / 500 items; projected \$2.35 to backfill the full 14k catalog).
2. **Embeddings are structurally weak at color.** FashionSigLIP is a CLIP-family model, and CLIP’s color–object binding is a documented failure mode. Relying on a vector to honor a color constraint is unreliable by design; a structured exact-match filter over the LLM color field is not.
3. **The product surface is an infinite scroll feed.** Success is “the first 20–60 cards are relevant,” not “the single best item is at rank 1.” This is a recall target under loose ordering, which a cascade (specific -> general) serves directly.

A proof of concept established that a flat Reciprocal Rank Fusion (RRF) of three channels (image cosine + LLM-text cosine + keyword) regresses on color queries versus the current ranker, because RRF rewards multi-channel agreement and the two correlated embedding channels outvote the one channel that honors the constraint. The cascade design removes this failure mode. Measured on the 500-item subset (§7), the cascade reaches on-attribute@10 = 1.00 and recall@10 = 1.00 on color queries, versus 0.83 / 0.96 for the current hybrid and the RRF prototype respectively.

1. Current State

1.1 Two keyword mechanisms, neither BM25

There is no Atlas Search in the stack. Lexical matching is implemented two different ways in two different code paths:

Path	Mechanism	Scoring
Browse / legacy search (home-controller.js)	MongoDB native \$text index on item_name + brand_name + description (clothes-schema.js), \$meta: textScore sort; regex substring for 1–2 char queries	TF-IDF (the legacy MongoDB text scorer)
Hybrid search (search-v4-controller.js -> hybrid-retrieval.js)	In-application word-boundary regex token match over a 2,000-item candidate pool; no text index used	Token-presence ratio (matched query tokens ÷ total query tokens)

The hybrid path is the more advanced of the two, but its keyword component is the weaker scorer: it measures only whether query tokens appear, with no inverse-document-frequency weighting and no term-frequency saturation. Neither path uses BM25.

1.2 Current hybrid ranker

searchClothesV4 runs a three-layer pipeline:

- **Layer A — brand alias expansion:** fp -> free people / freepeople, etc.
- **Layer B — spell correction:** distance-1 SymSpell, produces a corrected query variant.
- **Layer C — dual-query hybrid:**

$$\text{score} = 0.7 \cdot \text{cosine} + 0.3 \cdot \text{keyword} + 0.01 \cdot \text{popularity}$$

Cosine is computed in Node against a single fused embedding fetched per item; the query is encoded by an external Cloud Run encoder service. A color entity, when detected, hard-filters the candidate pool by regex.

1.3 Current embeddings

clothes_embeddings.embedding is a **single fused vector** ($0.5 \cdot \text{text} + 0.5 \cdot \text{image}$) per item. The dual-vector storage proposed in the embedding plan is not yet in production. The collection is schemaless (strict: false), so additional vector fields can be added without migration.

1.4 The failure this redesign targets

A query for a color (grey, cream, navy) ranks off-color items highly. Two root causes:

- The keyword blob includes brand_name, so grey matches the brand “Grey Lab” and surfaces a red Grey Lab dress.
- The fused embedding’s color signal is too weak to override, because color binds poorly in CLIP-family models.

The redesign removes brand from all lexical and embedding text, moves color to a structured filter over the LLM field, and ranks per intent.

2. Design Principles

1. **LLM attributes are the source of truth; seller free-text is, at most, weak context.** Search ranks and filters on the normalized LLM attribute set. The seller’s raw text contributes only a low-weight embedding signal and the brand string.
 2. **Structured attributes are constraints, not preferences.** A color or category in the query is a hard requirement. Finite, enumerable attributes (color, category, length) are enforced by exact-match filtering, not by hoping a vector honors them.
 3. **Optimize recall over an ordered feed, not precision at rank 1.** The target metric is recall@20 / recall@60 with relevant items anywhere in the feed. This is what a cascade naturally produces.
 4. **Avoid score fusion across heterogeneous signals.** Lexical, dense, and popularity scores are not comparable. Sections are concatenated by a fixed intent priority; ranking within a section uses a single comparable signal.
-

3. Why Prior Fusion Approaches Fall Short

3.1 Linear hybrid

$0.7 \cdot \text{cosine} + 0.3 \cdot \text{keyword}$ requires the two scales to be comparable. They are not: cosine sits in a narrow $[0.2, 0.8]$ band, keyword is near-binary. Any fixed weight is a compromise that holds for one query distribution and breaks on the next. This is the score-comparability problem.

3.2 RRF (proof of concept result)

A flat RRF of three channels was evaluated on the 500-item LLM subset. On the cream query:

Method	Top-1	Top-1 actual color	off-color @ 5
Current hybrid	Lioness Set	Cream	0
Flat RRF (img + llm_txt + keyword)	Funky	Grey	4
Keyword channel alone	Farm Rio Cream Maxi	Blue, Cream	3

The keyword channel alone ranked a cream item first (clean score separation). But RRF combines ranks via a concave function:

$$\text{rrf}(item) = \sum_c \frac{1}{60 + \text{rank}_c(item)}$$

so three middling appearances (≈ 0.043) beat one rank-1 plus two poor ranks (≈ 0.023). Because two of the three channels are FashionSigLIP cosines (image and text from the **same** model), they are correlated and jointly outvote the keyword channel, surfacing “visually cream-like but actually grey” items.

RRF is the correct tool when channels capture independent relevance dimensions. It is the wrong tool when one channel is authoritative for a query class (color) and the others are correlated noise for that class. The cascade design avoids fusing them at all.

4. Target Architecture: Intent-Gated Cascade

The engine produces **one concatenated, de-duplicated, scrollable list**, assembled from up to three sections in fixed priority order, filled to K (e.g. 60). Each section is ranked by the method appropriate to its intent.

```
query
|
v
query understanding (normalize, spell-correct, detect brand/color/category)
|
+-- Section 0 Brand (exact / fuzzy brand match)
+-- Section 1 Structured (color / category hard filter over LLM fields)
+-- Section 2 Semantic (brand-stripped dense retrieval; fills the feed)
|
v
dedupe -> concatenate [S0, S1, S2] -> top-K feed
```

4.1 Data model (LLM gold standard)

clothes_ai_attrs, one document per item:

```
{
  cloth_id,
  colors: ["cream", "white"],    // multi-value, normalized; multicolor => all listed
  color_primary: "cream",       // dominant color, for single-color tie-breaks
  category: "dress",
  subcategory: "shift",
  length: "short" | null,
  fit: "fitted" | null,
  occasions: ["cocktail", "summer"],
  material_guess: "cotton-blend" | null,
  listing_quality: "good" | "mid" | "bad",
  normalized_text: "cream short shift dress, fitted fit, for cocktail or summer, cotton",
  model_name, model_revision, extracted_at
}
```

Key points:

- **colors is a normalized multi-value array.** If the LLM identifies multiple colors, all are stored. A cream query matches any item whose colors contains cream (OR within the color facet).
- **A color-normalization map** collapses the LLM's free-form color vocabulary (~53 raw labels observed) into a canonical set (~20 entries; e.g. gray → grey, cobalt / royal blue → navy family). This map is also the vocabulary the query parser matches against. It is small enough to hand-review.
- **Brand never appears** in normalized_text or in any lexical blob — only in clothes.brand_name for the brand path and for display/filtering.

Embeddings (clothes_embeddings), three vectors per item, all included in dense fusion with fixed weights:

- llm_txt_embedding — FashionSigLIP over normalized_text. **Primary** text channel, weight **0.55**.
- img_embedding — FashionSigLIP image encoder. Weight **0.35**.
- seller_txt_embedding — FashionSigLIP over the raw seller description. **Low-weight backstop**, weight **0.10** — it is included (the seller's own words occasionally carry signal the LLM missed) but kept deliberately small because seller text is the untrusted source. Items with no description get a zero vector and contribute nothing. ~Half the catalog has a usable description (median ~35 chars), so this channel is sparse by nature.

4.2 Query understanding

parse(rawQuery):

```

q      = normalize(spellCorrect(rawQuery))    # lowercase, strip punctuation
brand  = brandMatch(q)                       # fuzzy, see §4.3
colors = tokens of q present in COLOR_VOCAB  # canonicalized
category = first token of q present in CATEGORY_VOCAB
occasions= tokens of q present in OCCASION_VOCAB
residual = q minus (brand, color, category, occasion) tokens
return {brand, colors, category, occasions, residual}

```

The four vocabularies (BRAND_SET, COLOR_VOCAB, CATEGORY_VOCAB, OCCASION_VOCAB) are not new infrastructure. They extend the vocabulary pipeline that already powers the current spell-correction and brand-alias layers. See §4.2.1 for exactly how they are built and refreshed.

4.2.1 Vocabulary provenance and refresh

This reuses the existing search-vocabulary mechanism end to end; only the data source for the attribute scopes changes.

Build (offline, RecSysPro). `src/features/search_vocab.py` already emits per-scope vocabulary rows from the nightly catalog snapshot. Today it produces five scopes — `items_brand`, `items_color`, `items_category`, `items_term`, `closets_name` — where `items_color` and `items_category` are derived from the **seller-provided** `color` and `category_titles` fields. These two scopes are exactly the polluted source this redesign moves away from.

For V6, the same module gains attribute scopes sourced from the **LLM clothes_ai_attrs** collection instead of seller fields:

- `COLOR_VOCAB` <- distinct values of `clothes_ai_attrs.colors`, canonicalized through the color-normalization map (§4.1).
- `CATEGORY_VOCAB` <- distinct `clothes_ai_attrs.category`.
- `OCCASION_VOCAB` <- distinct `clothes_ai_attrs.occasions`.
- `BRAND_SET` <- unchanged; the existing `items_brand` scope from `clothes.brand_name`, plus the `brand_synonyms.json` alias seed.

This is a handful of new functions reading one additional Mongo collection — no new job and no new scheduler.

Write + schedule. The vocabulary rows are wholesale-rewritten to the Mongo `search_vocab` collection (and `search_aliases` for brand) by the existing nightly Cloud Run Job `ecloset-recsys-build-prod` (`build_recsys_artifacts.py --mode full`). This job is triggered by the Cloud Scheduler entry `ecloset-recsys-build-prod-nightly` (cron `0 10 * * * UTC`, i.e. 3am PT), which is enabled and running in production. The V6 attribute scopes are produced inside this same run, immediately after the LLM extraction step (P2) populates `clothes_ai_attrs`.

Consume (online, API). `services/search/spell-correct.js` and `alias-lookup.js` already load these collections into an in-memory cache at startup and refresh on a `REFRESH_MS` interval (with coalesced reads). The V6 query parser reads the same cached vocabularies. No per-request database round trip; new attribute values become live within one cache-refresh interval after the nightly job writes them.

4.3 Section 0 — Brand

Triggered when the query matches a known brand.

- **Matching:** normalize (lowercase, strip punctuation/suffixes), then Damerau-Levenshtein with a **length-relative threshold** (~1 edit per 5 characters: fenty allows 1 edit, anthropologie allows 2). Damerau-Levenshtein counts adjacent transpositions as a single edit, which covers the large majority of real typos. Matching is on the whole query (or a contiguous brand span), never a partial substring of a longer ambiguous query.
- **Ranking:**
 - residual present (lululemon align leggings): BM25 over item_name + normalized_text + description on the residual tokens.
 - pure brand (grey lab): order by likeCount, then recency.

4.4 Section 1 — Structured

Triggered when the query contains a color and/or category token.

- **Hard pre-filter** over the LLM gold fields:
 - color: query colors must be present in item.colors.
 - category: item.category equals the query category, when present.
 - occasion: applied as a soft boost, not a hard filter (occasion labels are noisier and disagreement is higher).
- **Ranking the survivors:**
 - residual present (cream flowy dress): dense similarity (llm_txt_embedding weighted high, img_embedding added) within the filtered set.
 - pure attribute (cream): order by listing_quality, then likeCount, then recency — all survivors equally satisfy the constraint.

On a small catalog held in memory, this is exact: filter first, then run a brute-force cosine scan over the surviving subset. There is no approximate-index recall penalty.

4.5 Section 2 — Semantic tail

Always runs; fills the feed.

- **Candidates:** full catalog (brand-stripped embeddings), excluding items already shown in Sections 0–1.
- **Ranking:** dense fusion of the three channels against the full query — $0.55 \cdot \text{llm_txt} + 0.35 \cdot \text{img} + 0.10 \cdot \text{seller_txt}$ (the same fixed weights as §4.1; the seller term is zero for items with no description).
- **No keyword reranking** here — conceptual queries (y2k aesthetic, going out top) carry no lexical signal, and keyword reranking would degrade them. This section also catches any LLM color mislabels that the Section 1 hard filter excluded.

4.6 Ranking method by intent

Query class	Ranker	Rationale
Brand + residual	BM25 on residual	lexical signal is authoritative
Pure brand	likeCount / recency	no residual to rank
Structured + residual	dense within filtered set	concept ranks on-attribute items
Pure structured	listing_quality / likeCount	all survivors equally on-attribute
Conceptual	dense only	no lexical signal; keyword would hurt

Keyword/BM25 is used where lexical signal exists (brand, literal product, residual within a filtered set). It is never the universal final reranker, and it is never applied to the conceptual tail.

4.7 Optional precision step (V2)

A cross-encoder reranker (fashion-tuned or hosted) can later re-score the top ~30 of Section 2 to polish ordering. It is unnecessary for Sections 0–1, which are already precise, and optional for Section 2 given the recall-oriented feed target. Deferred until feed metrics indicate ordering, not relevance, is the bottleneck.

5. Lexical Layer: Options and Recommendation

The cascade needs a lexical ranker for the brand and structured-residual cases (§4.6). Three options, given the current stack (legacy \$text TF-IDF index + in-application regex; no Atlas):

Option	What it is	Operational cost	Quality
A. In-application BM25	rank_bm25 (or equivalent) over the candidate pool, in the encoder/search service	None — no new infrastructure	True BM25 (TF saturation + IDF + length norm)
B. MongoDB \$search (Lucene BM25)	The \$search aggregation stage, now available in MongoDB Community Edition	Requires deploying and operating the mongot search process alongside mongod; verify against the deployed MongoDB version	True BM25, index-backed, scales beyond in-memory

Option	What it is	Operational cost	Quality
C. Keep legacy \$text	The existing TF-IDF text index	None — already deployed	TF-IDF only; “not suitable for complex relevance tuning” per MongoDB docs

Decision: Option A (in-application BM25) for V6. At 14k items, BM25 over the (already bounded) candidate pool is sub-millisecond, needs no new infrastructure, and gives proper lexical ranking immediately. The legacy \$text index remains in place as a fast candidate-pool pre-filter. Option B (\$search BM25) is recorded as a future path — worth revisiting only if catalog scale or operational appetite later justifies running mongot, at which point it would replace both current keyword mechanisms with a single index-backed BM25. It is not adopted now.

6. Worked Examples

Query	Sections that fire	Behavior
grey lab	S0 only (+ S2 fill)	Resolves entirely to a brand; returns Grey Lab items by likeCount, then a similar-items tail. The color-brand collision is structurally impossible.
grey	S1 (+ S2 fill)	Hard-filter colors contains grey, rank by quality/likeCount; visually-grey fill below.
cream dress short	S1 (+ S2 fill)	Hard-filter colors contains cream AND category = dress AND length = short; dense-rank survivors.
lululemon align leggings	S0 (+ S2 fill)	Filter to Lululemon, BM25-rank by align leggings.
y2k aesthetic	S2 only	Pure dense retrieval; no lexical signal, no keyword reranking.

7. PoC Validation Results (P1)

All three retrieval methods were run on the 500-item LLM subset. Ground truth is the LLM gold-standard attributes (colors[], category, normalized brand); the question measured is which

method honors that contract and how completely. Placeholder/junk listings were filtered identically from all three methods. Reproduce with `python -m src.eval.search_v6_eval`.

7.1 Color queries

Mean over 7 single-color queries (cream, navy, white, black, pink, red, green):

Method	on-attribute@10	recall@10
V4 hybrid	0.83	0.83
V5 RRF	0.96	0.96
V6 cascade	1.00	1.00

The gap widens on the colors embeddings handle worst. Per-query on-attribute@10:

Query	V4	V5	V6
green	0.50	0.80	1.00
red	0.70	0.90	1.00
navy	0.80	1.00	1.00

7.2 Color + category queries

Mean over 4 queries (cream dress, navy dress, pink top, black set):

Method	on-attribute@10	recall@10
V4 hybrid	0.55	0.71
V5 RRF	0.63	0.77
V6 cascade	0.73	1.00

V6 recall@10 is 1.00 on every query — it surfaces every matching item. Its on-attribute@10 is below 1.00 only because some constraints have fewer than 10 matching items in the 500-sample (e.g. pink top has just 2), so the feed correctly continues with related items past the exhausted constraint set. On the well-populated constraints (cream dress, gold set 41; navy dress, gold set 20) V6 scores on-attribute@10 = 1.00.

7.3 Brand queries

Mean over 6 brand queries: V6 brand-section precision = 1.00 and recall@10 = 1.00 — every item placed in the brand section is the correct brand, and the entire brand inventory is surfaced. For brands with fewer than 10 items in the subset (most of them — e.g. lululemon and farmrio have 1 each), the feed fills the remaining slots with similar items, which is the intended small-brand behavior. (A naive precision@10 over the filled feed therefore reads low, but that is the metric penalizing a deliberately fuller feed, not a quality loss.)

7.4 Conceptual queries

Pure-conceptual queries (y2k aesthetic, cozy fall sweater, beachy summer vacation) route correctly to the semantic tail and, with junk listings filtered, return on-theme items. The cascade does not regress conceptual retrieval relative to the dense baseline.

7.5 Interpretation

V6’s on-attribute advantage on color is partly by construction — it hard-filters on the gold field. That is the point: V6 *guarantees* the expressed constraint while still filling the feed ($\text{recall@10} = 1.00$), whereas V4 and V5, which do not filter, leak off-constraint items (V4 surfaces 50% off-color on green). The PoC also surfaced a data-hygiene action item: ~placeholder “test” listings pollute the dense tail and should be filtered at index time (a known issue; filtered at query time here).

8. Implementation Plan

Phase	Repo	Work
P0 — color vocabulary	RecSys (PoC)	Build the canonical color-normalization map from LLM value-counts; hand-review (~50 rows). Store <code>colors[]</code> in <code>clothes_ai_attrs</code> .
P1 — cascade prototype	RecSys (PoC)	Implement query-understanding + three-section assembly; extend the eval harness with recall@K and on-attribute@K ; compare cascade vs current hybrid vs flat RRF on the 500-item subset.
P2 — full backfill	RecSysPro	Run LLM extraction over the full 14k catalog ($\approx \$2.35$); populate <code>clothes_ai_attrs</code> and the three-vector <code>clothes_embeddings</code> .

Phase	Repo	Work
P3 — API integration	E-Closet-Apis	Port query-understanding + cascade into services/search/; fuzzy brand match; structured pre-filter via Mongo query; dense ranking in the encoder service; in-app BM25 for lexical tiers. Feature flag SEARCH_V6_CASCADE.
P4 — A/B and metrics	Apis + RecSys	recall@20, on-attribute rate, zero-result rate, CTR vs the current hybrid.
P5 — precision polish (optional)	Apis	Cross-encoder rerank on the Section 2 top-30.

7.1 Self-hosted MongoDB notes

- Structured pre-filter (colors, category) is an ordinary indexed Mongo query — fast, exact, no Atlas dependency.
- Dense cosine runs in-process as an exact brute-force scan over the (pre-filtered) embedding matrix; 14k × 768 is trivial. No \$vectorSearch and no approximate index are required at this scale.
- Lexical ranking uses in-app BM25 (Option A) or, later, \$search BM25 (Option B) if mongot is deployed.

9. Metrics

Primary (offline, on a labeled query set):

- **recall@20 / recall@60** — fraction of relevant items appearing in the first 20 / 60 cards. The headline metric, replacing precision@1.
- **on-attribute rate @ K** — for color/category queries, fraction of the first K satisfying the stated constraint (should be ~100% within Section 1).
- **section coverage** — how often each section fires and how much of the feed it fills.

Secondary (online, post-launch):

- zero-result rate (should fall), search->card-click CTR, card->PDP rate, scroll depth, query-reformulation rate (a too-strict brand/facet threshold surfaces here).

10. Risks and Open Questions

1. **Intent-detection collisions.** A color word that is also a brand name — precedence rule: the brand path wins only on a whole-query match; a color token inside a longer query routes to the structured filter. Requires a small precedence test suite.
 2. **Color-vocabulary drift.** New color labels appear as the catalog grows; the normalization map needs a periodic refresh job.
 3. **Occasion as soft vs hard filter.** Proposed as soft; promotable to hard if precision requires it.
 4. **Section-ordering rigidity.** If users dislike a full brand block first, Section 0 can be interleaved with Section 2 after the first N items. Validate in A/B.
 5. **Lexical layer has no external dependency.** V6 uses in-application BM25 (Option A), so there is no `mongot / $search` prerequisite. Only revisit `$search` (Option B) if a future catalog scale outgrows in-memory ranking.
 6. **listing_quality in search.** ~84% of items rate mid. Use `listing_quality` only as a soft tie-break in search (and as the For You multiplier), never a hard filter — search is intentional discovery.
-

11. References

Hybrid search, RRF, and reranking:

- [Hybrid Search: BM25, Vector & Reranking 2026 — Digital Applied](#)
- [Hybrid Search + Reranking Playbook — OptyxStack](#)
- [Hybrid retrieval with RRF: score normalization — Chauzov](#)
- [The Critical Role of Rerankers in RAG — Medium](#)

E-commerce query understanding and intent routing:

- [Query Brand Entity Linking in E-Commerce Search — arXiv 2502.01555](#)
- [Hierarchical Query Classification in E-commerce — Amazon Science](#)
- [Query Understanding in Ecommerce — Wizzy](#)

CLIP color/attribute binding:

- [CLIP Behaves like a Bag-of-Words Cross-modally — arXiv 2502.03566](#)
- [Common Data Properties Limit Object-Attribute Binding in CLIP — arXiv 2507.07985](#)

Faceted / structured attribute search:

- [Faceted Search Best Practices for E-commerce \(2026\) — BrokenRubik](#)
- [Semantic Search vs Keyword Search — RapidSearch](#)

LLM attribute extraction:

- [Leveraging multimodal LLMs for Shopify's global catalogue — Shopify \(ICLR 2025\)](#)
- [BEATS: Bootstrapping E-commerce Attribute Taxonomies — arXiv 2606.04909](#)
- [Using LLMs for Extraction and Normalization of Product Attribute Values — arXiv 2403.02130](#)

Cascade and result merging:

- [Learning Cascade Ranking as One Network — arXiv 2503.09492](#)
- [Unified Learning-to-Rank for Multi-Channel Retrieval in E-Commerce — arXiv 2602.23530](#)

Fuzzy matching:

- [The Two Algorithms Every Developer Uses Wrong — Medium](#)
- [What is fuzzy search? — Typesense](#)

MongoDB full-text and filtering:

- [Assign Weights to \\$text Query Results \(self-managed\) — MongoDB Docs](#)
- [Text Search With MongoDB \(BM25 / TF-IDF\) — DEV](#)
- [Pre and Post Filtering in Vector Search with Metadata — DEV](#)